WEEK : 3

Seller and Inventory Management System

Scenario

The e-commerce system maintains information about sellers and inventory. The system stores seller details and tracks the stock quantity, stock status, and latest update date of products supplied by different sellers.

The system helps manage seller information and monitor product inventory to ensure that product stock and availability can be tracked effectively.

1. Creating Seller and Inventory Tables

The Seller and Inventory tables are created to maintain seller details and inventory information in the e-commerce system.

2. Establish Relationships Using Primary Key

Primary keys are used to uniquely identify records in the Seller and Inventory tables.

- Seller_ID is the primary key of the Seller table.
- Inventory_ID is the primary key of the Inventory table.

The Inventory table is connected to both the Product and Seller tables using foreign keys.

Seller → Inventory

One seller can supply multiple products. Therefore, the relationship between Seller and Inventory is One-to-Many (1:M).

Product → Inventory

A product can have inventory information associated with it. The Inventory table uses Product_ID to identify the corresponding product.

3. Foreign Key for Inventory Table

Foreign Keys

- Product_ID in the Inventory table references Product_ID in the Product table.
- Seller_ID in the Inventory table references Seller_ID in the Seller table.

4. Constraints

Unique Email ID

A UNIQUE constraint is added to the Email column in the Seller table. This ensures that two sellers cannot have the same email address.

Email NOT NULL

The NOT NULL constraint is applied to the Email column. This ensures that every seller record contains an email address.

Add Constraint to Check Stock Quantity is Positive

The CHECK constraint ensures that the stock quantity in the Inventory table is always greater than zero.

```
SQL> ALTER TABLE Seller
  2  MODIFY EMAIL VARCHAR2(100) NOT NULL;

Table altered.
```

```
SQL> ALTER TABLE Seller
  2  ADD CONSTRAINT UQ_SELLER_EMAIL UNIQUE (EMAIL);

Table altered.

SQL>
```

5. Inserting Values into Seller Table

The following values are inserted into the Seller table.

Seller Records:

Seller ID	Seller Name	Email	Phone	Address
1	Tech World	techworld@gmail.com	9876543210	Chennai
2	Mobile Hub	mobilehub@gmail.com	9876543211	Chennai
3	Digital Store	digitalstore@gmail.com	9876543212	Chennai

```
SQL>    INSERT INTO Seller (SELLER_ID, SELLER_NAME, EMAIL, PHONE, ADDRESS)
  2  VALUES (1, 'Tech World', 'techworld@gmail.com', '9876543210', 'Chennai');

1 row created.

SQL> INSERT INTO Seller (SELLER_ID, SELLER_NAME, EMAIL, PHONE, ADDRESS)
  2  VALUES (2, 'Mobile Hub', 'mobilehub@gmail.com', '9876543211', 'Chennai');

1 row created.

SQL> INSERT INTO Seller (SELLER_ID, SELLER_NAME, EMAIL, PHONE, ADDRESS)
  2  VALUES (3, 'Digital Store', 'digitalstore@gmail.com', '9876543212', 'Chennai');

1 row created.

SQL>
```

```
1 row created.

SQL> SELECT * FROM Seller;

 SELLER_ID
----------
SELLER_NAME
--------------------------------------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
PHONE
---------------
ADDRESS
--------------------------------------------------------------------------------
         1
Tech World
techworld@gmail.com

 SELLER_ID
----------
SELLER_NAME
--------------------------------------------------------------------------------
EMAIL
--------------------------------------------------------------------------------
PHONE
---------------
ADDRESS
```

6. Inserting Values to Inventory Table

The following values are inserted into the Inventory table.

Inventory ID	Product ID	Seller ID	Stock Quantity	Stock Status
1	1	1	50	Available
2	2	2	30	Available
3	3	3	20	Available

The inventory records are associated with the corresponding Product and Seller records using Product_ID and Seller_ID.

```
SQL> INSERT INTO Product (PRODUCT_ID) VALUES (1);

1 row created.

SQL> INSERT INTO Product (PRODUCT_ID) VALUES (2);

1 row created.

SQL> INSERT INTO Product (PRODUCT_ID) VALUES (3);

1 row created.

SQL> COMMIT;

Commit complete.

SQL>
```

```
SQL> SELECT * FROM Inventory;

INVENTORY_ID PRODUCT_ID  SELLER_ID STOCK_QUANTITY
------------ ---------- ---------- --------------
STOCK_STATUS
--------------------------------------------------------------------------------
LAST_UPDA
---------
           1          1          1             50
Available
22-SEP-26

           2          2          2             30
Available
22-SEP-26

INVENTORY_ID PRODUCT_ID  SELLER_ID STOCK_QUANTITY
------------ ---------- ---------- --------------
STOCK_STATUS
--------------------------------------------------------------------------------
LAST_UPDA
---------

           3          3          3             20
Available
22-SEP-26


SQL>
```

Report 1: Seller-Wise Product Report

Purpose

This report displays the number of products handled by each seller and the total available stock associated with the seller.

SQL Query

```
SELECT SELLER_ID,
    COUNT(PRODUCT_ID) AS TOTAL_PRODUCTS,
    SUM(STOCK_QUANTITY) AS TOTAL_STOCK
FROM Inventory
GROUP BY SELLER_ID;
```

Report 2: Stock Availability Report

Purpose

This report displays the product ID, stock quantity and current stock status.

SQL Query

```
SELECT PRODUCT_ID,
    STOCK_QUANTITY,
    STOCK_STATUS
FROM Inventory;
```

```
SQL> SELECT SELLER_ID,
  2         COUNT(PRODUCT_ID) AS TOTAL_PRODUCTS,
  3         SUM(STOCK_QUANTITY) AS TOTAL_STOCK
  4  FROM Inventory
  5  GROUP BY SELLER_ID;

 SELLER_ID TOTAL_PRODUCTS TOTAL_STOCK
---------- -------------- -----------
         1              1          50
         2              1          30
         3              1          20
```

```
SQL> SELECT PRODUCT_ID,
  2         STOCK_QUANTITY,
  3         STOCK_STATUS
  4  FROM Inventory;

PRODUCT_ID STOCK_QUANTITY
---------- --------------
STOCK_STATUS
--------------------------------------------------------------------
         1             50
Available

         2             30
Available

         3             20
Available
```

Report 3: Inventory Analysis

The Inventory Analysis report provides information about the overall inventory maintained in the system.

Total Available Products

The following query is used to count inventory records whose stock status is Available.

```
SELECT COUNT(*) AS TOTAL_AVAILABLE_PRODUCTS
FROM Inventory
WHERE STOCK_STATUS = 'Available';
```

Expected Result: 3

Product Out of Stock

The following query is used to count products whose stock status is Out of Stock.

```
SELECT COUNT(*) AS PRODUCTS_OUT_OF_STOCK
FROM Inventory
WHERE STOCK_STATUS = 'Out of Stock';
```

Expected Result: 0

Average Inventory Quantity

The following query is used to calculate the average inventory quantity.

```
SELECT AVG(STOCK_QUANTITY) AS AVERAGE_INVENTORY_QUANTITY
FROM Inventory;
```

Expected Result: 33.3333333

```
SQL> SELECT COUNT(*) AS TOTAL_AVAILABLE_PRODUCTS
  2  FROM Inventory
  3  WHERE STOCK_STATUS = 'Available';

TOTAL_AVAILABLE_PRODUCTS
------------------------
                       3
```

```
SQL> SELECT COUNT(*) AS PRODUCTS_OUT_OF_STOCK
  2  FROM Inventory
  3  WHERE STOCK_STATUS = 'Out of Stock';

PRODUCTS_OUT_OF_STOCK
---------------------
                    0
```

```
SQL> SELECT AVG(STOCK_QUANTITY) AS AVERAGE_INVENTORY_QUANTITY
  2  FROM Inventory;

AVERAGE_INVENTORY_QUANTITY
--------------------------
                33.3333333
```